Dataset Annotation & Label Studio

Module 1: Fundamentals of Data Annotation & Supervised Machine Learning

Detailed course notes: the original Module 1 material, expanded with extra explanations, worked examples, quality metrics, annotation formats, scaling strategies and a full answer key for self-testing.

What you will learn in this module

- What data annotation is, and how features, labels and ground truth fit together
- Why supervised machine learning cannot work without labeled data
- The annotation workflow end to end, from raw data to a clean exported dataset
- Annotation types for text, images, audio, video and time series, plus common file formats
- How to measure label quality (agreement, Cohen's kappa, IoU) and how to avoid common mistakes
- Ways to scale annotation: in-house teams, crowdsourcing, weak supervision, active learning, LLM help
- A first look at Label Studio, which the rest of the course uses

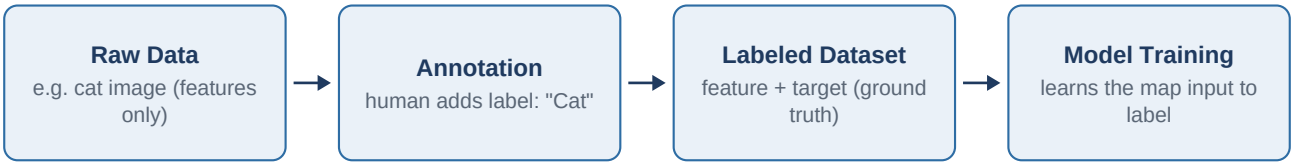
Contents

1. What is Data Annotation? . . . 2
2. Why is Data Annotation Needed? . . . 3
3. The Annotation Workflow (in detail) . . . 4
4. Worked Examples . . . 6
5. Annotation Types by Data Format . . . 7
6. Real-World Applications . . . 9
7. Guidelines, Quality and Inter-Annotator Agreement . . . 9
8. Common Mistakes and How to Fix Them . . . 11
9. Ways to Get Data Labeled (Scaling Annotation) . . . 12
10. A First Look at Label Studio . . . 12
11. Key Takeaways . . . 14
12. Revision Questions with Answers . . . 14
13. Further Reading and Sources . . . 15

1. What is Data Annotation?

Data annotation (also called data labeling) is the process of adding informative tags or labels to raw, unstructured data such as text, images, audio or video, so that a machine learning algorithm can learn from it. The labels can be simple (a single word such as *Cat*) or rich (a pixel-perfect outline of a tumor in an MRI scan).

In **supervised machine learning**, the algorithm learns from examples that contain two parts: the input data (called **features**) and the correct output (called the **label** or **target**). The trusted, human-verified set of labels is called the **ground truth**.



> **Remember**
> The model never learns from raw data alone. It always learns from data paired with its label. Annotation is the step that creates that pairing.

1.1 Key terms

Term	Meaning
Raw data	Original, unprocessed files: text, images, audio, video. No labels attached.
Features	The input the model looks at: the pixels of an image, the words of a sentence, the columns of a table row.
Label / Target	The correct answer attached to an input, for example "Cat" or "Positive". Often written as y, while the input is x.
Ground truth	The trusted, human-verified labels that the model treats as the correct answer. See the note below on what "truth" really means.
Annotator	The person who assigns labels to the data. Also called a labeler or rater.
Annotation guidelines	Written rules that tell annotators exactly how to label each case, including edge cases.
Labeled dataset	Raw data and its labels stored together, ready for training.
Taxonomy / Label schema	The complete set of allowed labels and how they relate (for example Positive / Negative / Neutral, or a hierarchy such as Vehicle > Car > Hatchback).
Gold standard (golden set)	A small set of examples labeled very carefully by experts. Used to test annotators and to evaluate models.
Inter-annotator agreement (IAA)	A measure of how often different annotators give the same label to the same item. A direct measure of label quality.
Pre-annotation	Labels suggested automatically (by a model or rules) that a human then reviews and corrects. Faster than labeling from scratch.
Adjudication	Resolving disagreements between annotators, usually by a senior reviewer or by majority vote.

Term	Meaning
Metadata	Extra information about an item or its annotation, such as source, timestamp, annotator ID or confidence.

> **Is ground truth really "true"?**
> Not always. Ground truth means the best available human agreement, not an absolute fact. Two doctors can outline the same tumor slightly differently, and two readers can disagree on whether a sentence is sarcastic. That is why we measure agreement between annotators and write clear guidelines: the goal is labels that are consistent and defensible, even if no label is perfect.

1.2 Where annotation fits among learning types

Learning type	Labels needed?	Idea	Example
Supervised	Yes, for every training example	Learn the mapping from input x to label y	Spam vs not spam; image classification
Unsupervised	No	Find structure in unlabeled data	Customer clustering
Semi-supervised	A few labels plus many unlabeled	Use labeled data to guide learning from unlabeled data	Labeling 1,000 images, training on 100,000
Self-supervised	No human labels; labels come from the data itself	Hide part of the input and predict it	Language models predicting the next word
Reinforcement learning	Rewards instead of labels	Learn by trial, error and reward	Game-playing agents

Even in the era of large pretrained models, human annotation is still essential. It is used to fine-tune models for a specific task, to build trustworthy test sets for evaluation, and to collect human preference data (ranking which answer is better) for aligning chatbots through techniques such as **RLHF** (Reinforcement Learning from Human Feedback).

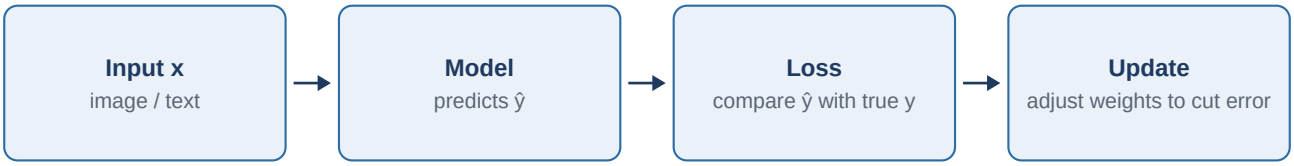
2. Why is Data Annotation Needed?

Machine learning algorithms cannot understand context from raw pixels or raw text files on their own. An image file is just an array of numbers. Without annotations marking that this group of pixels is a "car" or that group is a "house", an object detection model has no target output to learn from.

> **Core idea**
> Data annotation transforms raw files into structured training data. No labels means no supervision, and without supervision the model has nothing to learn from.

2.1 The technical reason: loss needs a target

A supervised model makes a prediction $\hat{y}$ for each input x. A loss function then compares the prediction with the correct label y and measures the error. The training algorithm (for example gradient descent) adjusts the model's parameters to reduce that error. If there is no y, there is nothing to compare against, so there is no error signal and the model cannot improve.



2.2 Beginner intuition: flash cards

Think of teaching a child with flash cards. Showing a picture alone teaches nothing. Showing the picture and saying its name is what makes learning possible. Annotation is how we write the name on the flash card for the machine.

2.3 Label quality decides model quality

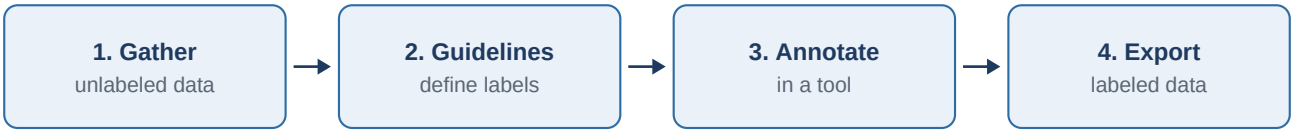
A model can only be as good as the labels it learns from. This is the old rule of "garbage in, garbage out". Many practitioners, most famously Andrew Ng, promote **data-centric AI**: instead of only tweaking model architectures, invest in making the data and labels consistent and correct. In many real projects, cleaning labels improves accuracy more than changing the model.

- Wrong labels teach the model wrong patterns (label noise).
- Inconsistent labels make similar inputs get different answers, so the model gets confused.
- Missing or biased labels make the model blind to, or unfair towards, some groups of cases.
- Good labels also give you a reliable test set, so you can trust your accuracy numbers.

> **Why it matters in practice**
> Data labeling is often one of the biggest costs and time sinks in an ML project, sometimes more than model training itself. Understanding annotation properly lets you plan the budget, choose tools and avoid expensive re-labeling later.

3. The Annotation Workflow (in detail)

The original notes describe four steps. In a real project each of these has smaller sub-steps. Here is the short version first, then the detailed one.



Step 1: Gather unlabeled data

Collect raw files such as CSV text rows, JPEG images, audio clips or JSON items. Think about whether the data really matches what the model will see in production. Include variety (different lighting, accents, writing styles, devices) and remove exact duplicates, corrupted files and private information (names, phone numbers, faces) where required. You rarely label everything: usually you label a well-chosen sample.

Step 2: Define annotation guidelines

Specify the tags or classes annotators may assign (for example Positive, Negative, Neutral) and write down exactly what each means. Good guidelines usually contain:

- A clear one-line task definition (what question is the annotator answering?).
- A definition for every label with 2-3 positive examples and 2-3 negative examples.
- Edge-case rules (sarcasm, mixed sentiment, blurry or cut-off objects, overlapping objects).
- What to do when unsure (use an Unsure/Other label, skip, or escalate to a reviewer).
- Tool instructions (how tight a bounding box should be, how to label occluded objects) and a version number with a change log.

> **Tip for beginners**
> Step 2 is the step most people rush, and it matters most. Clear guidelines are what keep different annotators giving the same label to the same data.

Step 3: Use an annotation tool

Load the raw files into an interactive interface, such as Label Studio, so annotators can assign labels quickly. Before the full project, run a **pilot**: have 2-3 annotators label the same small batch (say 50-100 items), compare results, find confusing rules, and update the guidelines. Then train annotators, and annotate at scale with a review process running alongside.

Step 4: Export labeled data

Save the annotations alongside the original data in a structured format (JSON, CSV, XML, or task-specific formats such as COCO or YOLO) ready for model training. Keep versions of the dataset (for example v1, v2) so that experiments can be reproduced.

The full loop in real projects

Stage	What happens	Output
Plan	Define the problem, success metric and label schema	Task definition, taxonomy
Collect and sample	Gather raw data, clean it, pick a representative sample	Raw data pool
Write guidelines	Document labels, examples and edge cases	Guidelines v1
Pilot	Small batch labeled by several people; measure agreement	Updated guidelines, IAA score
Annotate at scale	Main labeling, optionally with pre-annotations	Raw annotations
Quality control	Review samples, check agreement, use gold items, adjudicate disagreements	Verified labels
Export and version	Convert to training format, split into train / validation / test	Labeled dataset vX
Train and learn from errors	Train a model, find where it fails, label more of those cases	Next iteration (the loop repeats)

4. Worked Examples

4.1 Sentiment classification (text)

Suppose you have this unlabeled sentence: *"The battery life on this laptop is fantastic!"*

Item	Value
Raw data	The sentence string itself.
Annotation task	Sentiment classification (single label out of Positive / Negative / Neutral).
Human action	The annotator clicks Positive.

The resulting labeled record in JSON:

```
{
  "text": "The battery life on this laptop is fantastic!",
  "label": "Positive"
}
```

Here **text** is the feature and **label** is the target. Thousands of such records together form the labeled dataset used to train a sentiment model.

4.2 Named Entity Recognition, NER (text)

In NER, annotators highlight spans of text and give each span a type. Take the sentence *"Sundar Pichai is the CEO of Google in California."* The annotator marks three entities:

Span text	Type	Start (char)	End (char)
Sundar Pichai	PERSON	0	13
Google	ORGANIZATION	28	34
California	LOCATION	38	48

The same annotation can be stored token by token using the **BIO scheme**: **B-** marks the beginning of an entity, **I-** marks inside the entity, and **O** marks tokens outside any entity.

Token	Sundar	Pichai	is	the	CEO	of	Google	in	California	.
Tag	**B-PER**	**I-PER**	O	O	O	O	**B-ORG**	O	**B-LOC**	O

4.3 Bounding box (image)

For object detection, the annotator draws a rectangle around each object and picks its class. A single image might be stored like this (simplified, COCO-style, where bbox = [x, y, width, height] in pixels from the top-left corner):

```
{
  "image": "street_001.jpg",
  "annotations": [
    {"label": "pedestrian",    "bbox": [120, 80, 60, 150]},
    {"label": "traffic_light", "bbox": [300, 20, 30, 70]}
  ]
}
```

5. Annotation Types by Data Format

Data type	Typical annotation types
Text	Classification (sentiment, topic, intent, spam), multi-label tagging, named entity recognition, relation tagging, question-answer span selection, translation / summary quality rating, ranking of model answers.
Images	Classification, bounding boxes, polygons and segmentation, keypoints (for example human pose, facial landmarks), brush masks, image captioning.
Audio	Transcription (speech to text), speaker labeling (diarization), sound event tagging, emotion or intent labeling.
Video	Object tracking across frames, action recognition, temporal segments (start and end time of an event).
Time series / sensor data	Marking anomalies or events on a signal, labeling regions (for example normal / fault), activity recognition from sensors.
Tabular data	Labeling rows with a class (fraud / not fraud), validating or correcting values.

5.1 Types of classification labels

Type	Meaning	Example
Binary	Exactly two classes	Spam / Not spam
Multi-class	One label out of three or more	Positive / Negative / Neutral
Multi-label	Several labels can apply at once	A movie that is both Comedy and Romance
Hierarchical	Labels arranged as parent and child	Animal > Dog > Labrador

5.2 Image annotation: how precise?

Method	What the annotator draws	Precision	Used for
Classification	One label for the whole image	Lowest effort	"Is this a cat?"
Bounding box	Rectangle around each object	Medium	Object detection (cars, people)
Polygon	Outline following the object's shape	High	Irregular objects
Semantic segmentation	Every pixel gets a class	Very high	Road vs sidewalk vs sky; tumor regions
Instance segmentation	Every pixel gets a class and an object identity	Very high	Separating each person in a crowd
Keypoints	Points at body joints or landmarks	Medium	Pose estimation, face landmarks

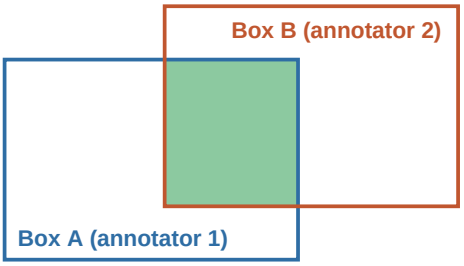
5.3 Common bounding-box file formats

Different tools and model libraries store boxes differently. Mixing these up is a classic source of bugs.

Format	Box representation	Units	Notes
COCO (JSON)	[x, y, width, height]	Pixels	x, y is the top-left corner. Very widely used.
Pascal VOC (XML)	xmin, ymin, xmax, ymax	Pixels	Two corners. One XML file per image.
YOLO (TXT)	class x_center y_center width height	Normalized 0 to 1	One text file per image, one line per object.

5.4 Measuring box agreement: IoU

Intersection over Union (IoU) measures how much two boxes overlap. IoU is 1 when the boxes match exactly and 0 when they do not overlap at all. It is used in two ways: to check how well two annotators agree, and to judge whether a model's predicted box counts as correct (a common threshold is IoU of at least 0.5).



Overlap (intersection) = green area

Union = area A + area B - overlap

IoU = overlap area / union area

IoU = 1: boxes match exactly. IoU = 0: no overlap.

5.5 Annotation for modern LLMs

- **Instruction data:** humans write good prompts and ideal answers for fine-tuning.
- **Preference ranking:** annotators compare two model answers and choose the better one (used in RLHF).
- **Safety and red-teaming labels:** marking harmful, biased or unsafe outputs.
- **Evaluation:** rating answers for correctness, helpfulness and faithfulness to sources.

6. Real-World Applications

Industry	Annotation task	What the annotator does
Autonomous driving	Object detection, lane and road segmentation	Draws bounding boxes around pedestrians, traffic lights and vehicles in street-view photos; marks lanes and drivable area.
Healthcare	Segmentation, classification	Outlines the exact region of tumors in MRI scans; labels X-rays as normal or abnormal. Requires medical experts.
Customer service	Text classification, intent detection	Sorts incoming support emails into categories such as Refund Request or Technical Issue; marks urgency.
E-commerce / retail	Product tagging, image classification	Labels product images by category, color and style; checks search relevance.
Finance	Text and tabular classification	Labels transactions as fraud or legitimate; extracts key fields from invoices and contracts (NER).
Agriculture	Image classification, segmentation	Marks diseased leaves, weeds and fruit ripeness in drone or phone photos.
Social media / content moderation	Text and image classification	Flags hate speech, spam or unsafe images according to policy.
Voice assistants	Audio transcription, intent labeling	Transcribes speech and labels what the user wants (set alarm, play music).
Satellite / geospatial	Segmentation, object detection	Outlines buildings, roads, crops and flood areas.

> **Domain expertise matters**
> Simple tasks (is there a cat in this photo?) can be done by almost anyone after short training. Specialized tasks (medical scans, legal text, industrial faults) need subject-matter experts, which makes them slower and more expensive. Choose annotators to match the difficulty of the task.

7. Guidelines, Quality and Inter-Annotator Agreement

7.1 Why quality control is needed

Annotators are human. They get tired, interpret rules differently, make slips, and sometimes rush. Quality control is the set of checks that catches these problems early, before the model learns from them.

7.2 Common quality techniques

- **Review / spot check:** a second person re-checks a random sample (for example 5-10 percent) of each annotator's work.
- **Overlap (redundant labeling):** the same item is labeled by 2-3 people; disagreements are sent to an expert.
- **Majority vote / consensus:** the label chosen by most annotators becomes the final label.

- **Gold items (honeypots):** items with known correct answers are secretly mixed in; annotators who fail them get retrained or removed.
- **Calibration sessions:** the team discusses tricky examples together and updates the guidelines.
- **Feedback loop:** train a model, look at where it disagrees with the labels, and re-check those items; many are label errors.

7.3 Inter-annotator agreement

The simplest measure is **percent agreement**: the share of items on which two annotators gave the same label. But it can be misleading, because two people guessing at random will still agree sometimes. **Cohen's kappa** fixes this by subtracting the agreement expected by chance:

Cohen's kappa (two annotators)

$$\textbf{kappa} = (p_o - p_e) / (1 - p_e)$$

p_o = observed agreement (share of items where both gave the same label)
p_e = agreement expected by chance, calculated from how often each annotator uses each label

Worked example. Two annotators label 100 reviews as Positive or Negative. They both say Positive on 40 items, both say Negative on 40 items, A says Positive but B says Negative on 10 items, and A says Negative but B says Positive on 10 items.

Quantity	Calculation	Result
Observed agreement p_o	(40 + 40) / 100	0.80
A says Positive	(40 + 10) / 100	0.50
B says Positive	(40 + 10) / 100	0.50
Chance agreement p_e	(0.5 x 0.5) + (0.5 x 0.5)	0.50
Kappa	(0.80 - 0.50) / (1 - 0.50)	**0.60**

Percent agreement looked high at 80 percent, but after removing chance the real agreement is 0.60, which is only moderate. That is the value of kappa.

Interpreting kappa (Landis and Koch scale)

Kappa value	Level of agreement
Below 0	Poor (worse than chance)
0.00 to 0.20	Slight
0.21 to 0.40	Fair
0.41 to 0.60	Moderate
0.61 to 0.80	Substantial
0.81 to 1.00	Almost perfect

Other measures: **Fleiss' kappa** for more than two annotators, and **Krippendorff's alpha**, which handles many annotators, missing labels and different data types. For boxes and segmentation masks, use IoU or the Dice coefficient. For text spans (NER), use span-level F1 between annotators. There is

no universal pass mark: tasks that are naturally subjective (sarcasm, toxicity) will always have lower agreement than clear tasks (is there a car in the image?).

> **Low agreement is a signal, not a failure**
> If kappa is low, the usual cause is unclear guidelines or an ambiguous label set, not lazy annotators. Look at the items they disagree on, find the pattern, and fix the rule.

8. Common Mistakes and How to Fix Them

Mistake 1: Inconsistent labeling guidelines

Not defining edge cases leads to noisy training data. Example: does sarcasm count as Negative or Neutral? If two annotators decide differently, the model receives conflicting answers for similar inputs. **Fix:** write explicit edge-case rules with examples, run a pilot, and update the guidelines.

Mistake 2: Confusing unlabeled data with raw data

Thinking raw data can be fed straight into a supervised model without target labels. Supervised models need the labels; the raw files alone are not enough. **Fix:** remember that raw data is only the input half of the training pair.

More pitfalls worth remembering

Pitfall	Why it hurts	How to handle it
Skipping a review step	Even careful annotators make errors that silently reach the model.	Spot-check a random sample; use overlap on a subset.
Class imbalance	If 95 percent of labels are one class, the model may learn to always predict that class and still look 95 percent accurate.	Collect more rare-class data, resample or use class weights, and evaluate with precision, recall and F1, not accuracy alone.
Changing rules mid-project	Older labels no longer follow the new rules and become inconsistent.	Version the guidelines; re-check or re-label old data after a change.
Ambiguous or overlapping labels	Annotators cannot tell "Complaint" from "Negative feedback".	Merge or redefine similar labels; make labels mutually exclusive when needed.
Too many classes too early	Annotation becomes slow and agreement drops.	Start with a simple schema; split labels later if needed.
Annotator fatigue and rushing	Accuracy falls over long sessions.	Short sessions, realistic targets, gold items to monitor quality.
Annotator or dataset bias	The model copies human or sampling bias (for example only daytime photos).	Use diverse annotators and diverse data; audit results by subgroup.
Data leakage	The same or near-identical item appears in both train and test sets, giving falsely high scores.	Remove duplicates; split by source (for example by patient or by user), not by random row.
Ignoring privacy	Personal data (faces, names, medical details) can be exposed to annotators or leaked.	Anonymize, restrict access, follow laws and consent rules.

Pitfall	Why it hurts	How to handle it
Never fixing wrong labels	Label noise stays in the dataset forever.	Use model disagreement and label-cleaning tools such as Cleanlab (confident learning) to find suspicious labels, then re-review.

9. Ways to Get Data Labeled (Scaling Annotation)

Approach	How it works	Good for	Watch out for
In-house team	Your own staff or domain experts label the data	Sensitive data, expert tasks, tight quality control	Slow and costly to scale
Outsourcing / vendors	A specialist company provides trained annotators	Large volumes with managed quality	Needs strong guidelines and QA on your side
Crowdsourcing	Many online workers (for example Amazon Mechanical Turk) do small tasks	Simple tasks, fast and cheap at large scale	Variable quality; needs gold items and overlap
Programmatic / weak supervision	Write labeling rules (heuristics) that label data automatically, then combine the noisy rules (for example Snorkel)	Huge unlabeled datasets, text tasks	Rules are noisy; validate on a hand-labeled set
Model-assisted (pre-annotation)	A model proposes labels; humans correct them	When a rough model already exists	Annotators may over-trust the suggestions (automation bias)
Active learning	The model picks the most uncertain or informative items for humans to label first	Saving labeling budget	Needs a training loop set up
Synthetic data	Generate artificial data with labels already known (simulation, generation)	Rare or dangerous cases	May not match real-world data
LLM-assisted labeling	A large language model labels or drafts labels; humans verify a sample	Text tasks at scale	Can be confidently wrong; always measure against a human gold set

> **Practical rule of thumb**
> Label a small, high-quality gold set by hand first. Then use it to test every shortcut (weak rules, model suggestions, LLM labels, crowd workers). If a shortcut does not match your gold set, it is not ready to use at scale.

10. A First Look at Label Studio

Label Studio is a popular open-source data labeling tool, developed by HumanSignal (previously Heartex). It runs in your browser and supports many data types (text, images, audio, video, time series, and more) through one flexible interface. You will use it throughout this course.

10.1 Why use a tool at all?

- **Fast and consistent:** clear buttons, shortcuts and a uniform interface for every annotator.
- **Customizable:** you describe your labeling task, and the interface is built from that description.
- **Collaboration:** multiple annotators, review of annotations, and agreement statistics.
- **Import and export:** data goes in as JSON, CSV, text, images and so on; annotations come out in formats ready for training.
- **Machine-learning integration:** you can connect a model that supplies pre-annotations, so humans only correct.

10.2 Core concepts you will meet

Concept	Meaning
Project	A container for one labeling job: its data, its labeling interface and its annotations.
Task	One item to be labeled (one sentence, one image, one audio clip).
Labeling configuration	An XML-like description of the interface: what data is shown and which labels or tools the annotator gets.
Annotation	The result a human produces for a task.
Prediction / pre-annotation	A label suggested by a model that the annotator can accept or fix.
Export	Download of the finished annotations, in formats such as JSON, CSV, COCO, YOLO, Pascal VOC or CoNLL (formats vary by data type).

10.3 A tiny labeling configuration (sentiment)

This is how the sentiment example from Section 4.1 would be described. Do not worry about the details yet; they are covered in later modules.

```
<View>
  <Text name="text" value="$text"/>
  <Choices name="sentiment" toName="text" choice="single">
    <Choice value="Positive"/>
    <Choice value="Negative"/>
    <Choice value="Neutral"/>
  </Choices>
</View>
```

After a human picks a choice, the exported record looks roughly like this (simplified):

```
{
  "data": {"text": "The battery life on this laptop is fantastic!"},
  "annotations": [{
    "result": [{
      "from_name": "sentiment", "to_name": "text", "type": "choices",
      "value": {"choices": ["Positive"]}
    }]
  }]
}
```

Getting started (preview for the next module)
Label Studio can be installed with Python's pip (`pip install label-studio`) and started from the terminal (`label-studio start`). It then opens in your browser, usually at `http://localhost:8080`. Installation details are covered in the next module; always follow the official documentation for the current steps.

11. Key Takeaways

- Supervised ML requires ground truth data created by human annotation.
- Annotation adds structured tags (labels, bounding boxes, categories, spans) to unstructured raw files.
- Models learn by comparing predictions with labels; without labels there is no error signal and no learning.
- The workflow is: gather data, define guidelines, annotate in a tool, export labeled data, and then review and iterate.
- Clear guidelines that cover edge cases are the best protection against noisy labels.
- Measure quality with review, gold items and agreement scores (percent agreement, Cohen's kappa, IoU).
- Watch for class imbalance, bias, data leakage and rule changes mid-project.
- Scale smartly: pre-annotation, active learning, weak supervision and LLM help are useful, but always check them against a human-labeled gold set.
- Label Studio is a flexible open-source tool that turns raw data into labeled datasets.

12. Revision Questions with Answers

Try each question yourself first, then compare with the answer.

Q1. What is the difference between features and ground truth?

Answer: Features are the inputs the model looks at (pixels, words, columns). Ground truth is the trusted, human-verified correct label for that input. The model sees features and learns to predict the ground truth.

Q2. Why can a model not learn from raw images alone?

Answer: A raw image is just an array of numbers. Without labels there is no correct target to compare predictions with, so the loss function cannot compute an error and the model has no learning signal.

Q3. List the four steps of the annotation workflow in order.

Answer: 1) Gather unlabeled data, 2) define annotation guidelines, 3) annotate using a tool such as Label Studio, 4) export the labeled data in a structured format (JSON, CSV, XML).

Q4. Why are edge cases such as sarcasm important to define in guidelines?

Answer: If edge cases are not defined, different annotators will label them differently. The model then receives conflicting labels for similar inputs, which creates noisy training data and lowers accuracy.

Q5. Give one example of an annotation task from healthcare and one from autonomous driving.

Answer: Healthcare: outlining (segmenting) tumor regions in MRI scans. Autonomous driving: drawing bounding boxes around pedestrians, traffic lights and vehicles in street-view images.

Q6. What is the difference between raw data and a labeled dataset?

Answer: Raw data is the original unlabeled files. A labeled dataset stores the raw data together with its labels, so it is ready for supervised training.

Q7. What does Cohen's kappa measure, and why is it better than percent agreement?

Answer: It measures agreement between two annotators after removing the agreement expected by chance. Percent agreement can look high even when much of it is luck, especially with few classes.

Q8. In a dataset with 95 percent Class A and 5 percent Class B, why is 95 percent accuracy not impressive?

Answer: A model that always predicts Class A would get 95 percent accuracy without learning anything useful about Class B. Use precision, recall and F1, and fix the imbalance.

Q9. What is the difference between COCO and YOLO bounding box formats?

Answer: COCO uses [x, y, width, height] in pixels from the top-left corner. YOLO uses class, x_center, y_center, width, height, all normalized between 0 and 1.

Q10. What is pre-annotation and what is one risk of it?

Answer: Pre-annotation means a model or rules suggest labels that a human reviews and corrects. Risk: annotators may trust the suggestions too much and accept wrong labels (automation bias).

Q11. Why should you keep a gold standard set?

Answer: It is a small, expert-verified set used to test annotator accuracy, check shortcuts such as weak rules or LLM labels, and to evaluate the final model fairly.

Q12. Name two annotation types for text and two for images.

Answer: Text: sentiment classification, named entity recognition. Images: bounding boxes, semantic segmentation (also polygons, keypoints).

13. Further Reading and Sources

Good places to deepen your understanding. Each is a well-known, reliable source on the topics in this module.

- **Label Studio official documentation** (labelstud.io): installation, labeling configuration, import and export formats, ML backend.
- **Andrew Ng,** Data-Centric AI talks and the DeepLearning.AI Data-Centric AI resources: why improving data quality can beat changing models.
- **Cohen (1960),** "A coefficient of agreement for nominal scales": the original kappa paper. Landis and Koch (1977) give the interpretation scale.
- **Lin et al. (2014),** Microsoft COCO: Common Objects in Context: the COCO dataset and annotation format. Pascal VOC and YOLO documentation for the other box formats.
- **Northcutt, Jiang and Chuang (2021),** Confident Learning, and the Cleanlab library: finding label errors automatically.

- **Ratner et al.,** Snorkel: programmatic labeling and weak supervision.
- **Ouyang et al. (2022),** Training language models to follow instructions with human feedback: how human preference labels are used in RLHF.
- **Settles (2009),** Active Learning Literature Survey: a classic overview of choosing which data to label.

> **Tip for revision**
> After finishing this module, try labeling 20 of your own sentences with three sentiment labels, write 5 edge-case rules, and ask a friend to label the same 20. Then calculate percent agreement and kappa. You will feel the importance of guidelines immediately.